Contents

CIM for Deep Learning — Step-by-Step Runbook
- How to use this document
- No claims until there is data
- Two boards, one design
- Diagrams
- The map
- STAGE 0 — Setup and resources (Weeks 1–2)
- STAGE 1 — Real engine, first layer, tested part by part (Weeks 1–13) ← START HERE
- STAGE 2 — Real data and real model (Weeks 3–11)
- STAGE 3 — Scale the engine and build the baseline (Weeks 8–15)
- STAGE 4 — FPGA implementation and board (Weeks 12–17)
- STAGE 5 — HLS and Cadence (Weeks 3–19)
- STAGE 6 — Measurement, then claims (Weeks 13–17)
- STAGE 7 — Demo polish, report, defence (Weeks 6–20)
- The critical path, at a glance
- Which LLM for what
- Data sources (verify before citing)
- Monday morning

CIM for Deep Learning — Step-by-Step Runbook

Project: 2023-FYDP-25 · UET Lahore · Supervisor: Prof. Dr. Muhammad Tahir **Calendar assumption:** Week 1 starts Monday 12 Oct 2026. **Planned work finishes in Week 20 (ending Sunday 28 Feb 2027), including the report and defence.** Weeks 21–24 (to Sunday 28 Mar 2027) are held as buffer: end of March is the hard limit, and nothing is planned into those weeks. Mark your exam and holiday weeks in the tracker and slide the plan around them. The estimates assume about 25 hours per person per week (the team's stated capacity), and one effort day means 6 focused hours.

How to use this document

Every step has the same shape:

- **Goal**: what this step proves.
- **Do**: the actions, in order.
- **Output**: the exact files or artifacts that must exist when you finish.
- **Review gate**: a checklist that someone *other than the author* runs. Every box must pass before the next step starts.
- **If it fails**: the first thing to try.
- **LLM help**: which model does what for this step.

The second rule: every part is tested on its own the day it is finished, and on the board as soon as there is a way to get it there.

The one rule: a step is not done because the author says so. It is done when the reviewer ticks every box in the review gate and shows the output in the weekly meeting.

Steps are numbered by stage. Within a stage, do them in order. Where two stages can run at the same time (you have four people), the "Lane" note says so. Who takes which lane is your call.

No claims until there is data

Nothing in this project is claimed until it has been measured. That gives two kinds of statement, and they must never be mixed up.

- **Design targets.** These are things you build *to*. You control them, and they are either met or not met. Examples: "bit-exact with the integer reference," "zero DSP slices in the CIM tile array," "≥ 8 MACs per cycle," "clock ≥ 50 MHz."
- **Hypotheses.** These are things you *test*, and you don't know the answer yet. Example: "the CIM engine's latency scales better with MAC count than an equivalently provisioned conventional engine." You don't know whether that's true, by how much, or where the crossover sits. Stage 6 measures it. The claim is written *after* that, in words the data supports.

No speedup number belongs anywhere in the synopsis, the slides or the report until Step 6.4 has produced it.

Two boards, one design

You will develop on the **Nexys A7-100T** and finish on the **ZCU104**. They are different kinds of device, and the plan treats them that way.

	Nexys A7-100T	ZCU104
Vivado part	`xc7a100tcsg324-1`	`xczu7ev-ffvc1156-2-e`
Family	Artix-7 (FPGA only)	Zynq UltraScale+ MPSoC (FPGA fabric plus Arm processors)
LUTs	63,400	230,400
Block RAM	4,860 Kb	312 × 36 Kb, plus 96 UltraRAM
DSP slices	240	1,728
Memory on board	128 MiB DDR2	2 GB DDR4 on the processor side; a DDR4 SODIMM connector for the fabric
Pmod ports	4	3
Microphone	onboard PDM microphone	none

(Nexys figures from Digilent's Nexys A7 page; ZCU104 figures from the AMD/Xilinx kit listings. Confirm both against the datasheets before you size anything.)

Role of each board.

- **Nexys A7-100T: development platform.** Stage 1 (the first real layer) happens here. It is a plain FPGA, so the design is fabric-only and talks to the PC over the board's USB-UART.
- **ZCU104: final platform.** It has far more fabric, so the MAC-count sweep can go higher, and it is where the final demo runs. Its fabric is reached through the Arm processor system, which adds a wrapper to build (Step 1.11).

Four rules that keep two boards from doubling the work:

1. **The core is board-agnostic.** `rtl/core/` (tile, engine, baseline, requantization, counters) contains nothing board-specific. Board wrappers live in `rtl/board/nexys/` and `rtl/board/zcu104/` and stay thin: clocks, pins, UART or AXI.
2. **Design to the smaller board first.** If the model and engine fit the Nexys, they fit the ZCU104. The ZCU104 then extends the sweep to larger MAC counts; it does not change the model.
3. **Never mix numbers from the two boards in one comparison.** Cycle counts are the same on both, because they come from the design. Clock frequency, LUTs and timing are not. Every row in `runs.csv` carries a `board` column, and every figure names its board.
4. **The measurement platform is a decision, not an assumption.** It is made in Week 13, after layer 1 has run on both boards (Decision D14).

Optional stretch, not scheduled: a DRAM-bound baseline on the ZCU104. At your model size all weights fit on-chip, so no real memory wall exists. The ZCU104's processor-side DDR4 would let a second baseline fetch weights from off-chip memory. It needs the processor-to-fabric data path and is real extra work. Decide at the Week 12 go/no-go (together with Genus) whether to attempt it. The runbook is complete without it.

Diagrams

Diagrams are review tools, not decoration. A datapath drawn before the RTL gives the reviewer something to check the code against, and it is what you will read when a testbench fails at 1 a.m. Every diagram below is the output of a step, and each is checked by someone who did not draw it.

Diagram	Drawn in	Must show
Tile datapath	Step 1.6 (before the RTL)	Weight storage per row, the AND mask, the per-column adder tree, shift-accumulate with the MSB subtract, **bit width on every wire**, pipeline registers
Bit-serial timing	Step 1.6 (before the RTL)	The 8 cycles, which bit-plane is broadcast when, when the accumulator is valid
Layer-1 engine block diagram and controller state machine	Step 1.8	Tile, input and output buffers, window addressing, requantization, the position and chunk loops, cycles per state
Layer-to-tile mapping	Step 3.1	A conv layer split into 16-row chunks, which tile and slot holds which weights, ping-pong buffers
CIM vs baseline, side by side	Step 3.2	The same MAC array in both; the only difference is where weights come from
System block diagram	Step 4.4 (draft in 3.1)	Microphone, I²S receiver, event detector, ring buffer, engine, host link, **clock-domain crossing marked**, both board wrappers
Verification ladder	Step 1.2	Float → integer reference → tile model → RTL sim → post-synthesis → board, with each pass criterion

The side-by-side CIM-versus-baseline diagram is the most important one for your viva. It makes the fairness argument visible in a way no paragraph does, and drawing it is part of the Step 3.2 fairness review.

How to work.

1. **Sketch first, then write RTL, then redraw.** The rough sketch guides the code; the clean version is redrawn after the tests pass so it matches the code exactly. A diagram that disagrees with the RTL is worse than no diagram.
2. **Every block is a module, and every module is a block.** This is a review check, run by a non-author.
3. **Conventions:** bit widths on every wire; registers drawn differently from combinational logic; clock domains in different colours; signal names identical to the RTL.
4. **Tools:** draw.io (free). Keep the editable `.drawio` file in `docs/diagrams/` and export **SVG or PDF**, never screenshots, so the report figures stay sharp. WaveDrom for timing diagrams.
5. **LLMs draft, you verify.** A model will produce draw.io XML, Mermaid or WaveDrom that looks right and has wrong bit widths or a missing MSB subtract. Check every model-drawn diagram against `arith_spec.md`.

The map

```
STAGE 0  Setup & resources              Weeks 1–2     everyone
STAGE 1  Real engine, first layer       Weeks 1–13    ← YOUR STARTING POINT
STAGE 2  Real data & real model         Weeks 3–11    Lane: runs alongside Stage 1
STAGE 3  Scale the engine + baseline    Weeks 8–15
STAGE 4  FPGA implementation & board    Weeks 12–17
STAGE 5  HLS + Cadence                  Weeks 3–19    Lane; only 5.1, 5.2, 5.6 are core
STAGE 6  Measurement → claims           Weeks 13–17
STAGE 7  Demo polish, report, defence   Weeks 6–20    then Weeks 21–24 = buffer
```

Why the starting point is the first real layer

The most common way projects like this fail: everyone builds their own piece at full scale, and the pieces are connected for the first time in month 6. Then nothing agrees with anything, and it takes weeks to find out whether the bug is in the quantization, the reference model, the RTL, the synthesis or the UART.

This plan avoids that by testing every part as soon as it is ready, and by getting one real layer through the **whole chain** (model → quantization → integer reference → tile → engine → synthesis → board → UART → result) by Week 8. By then:

- every interface is defined and tested (file formats, rounding modes, the UART protocol);
- the board link works, and has been tested on its own;
- one real layer of your network is bit-exact on the board.

From then on, Stage 3 adds the remaining layers one at a time, each tested in simulation and on the board before the next. Nothing built in Stage 1 is thrown away.

STAGE 0 — Setup and resources (Weeks 1–2)

Step 0.1 — Resource inventory with Sir

Week 1 · 1–2 days · Needs: nothing

Goal. Know exactly what hardware and tools you have before planning around them.

Do. Get written answers to these questions:

1. **FPGA boards:** confirm the Nexys A7-100T (`xc7a100tcsg324-1`) and the ZCU104 (`xczu7ev-ffvc1156-2-e`). For each: is it ours for the year, or shared in the lab? Can it leave the lab? **When is the ZCU104 available to you?**
2. **Cadence: which tools exactly?** Ask for the licence list. The ones that matter to you:
   - Xcelium (simulator)
   - JasperGold (formal verification)
   - Conformal (equivalence checking)
   - Genus (logic synthesis)
   - Innovus (place and route)
   - Stratus HLS (high-level synthesis)
   - Joules (power)
3. **Cadence: which process design kit (PDK) and standard-cell library?** For example Cadence's generic 45 nm (GPDK045) or a foundry kit through Europractice. *Without a PDK, Genus and Innovus are useless to you.* Is there an SRAM memory compiler?
4. **Cadence access:** lab machines only, or remote server? Hours? Is there anyone who has run the flow before?
5. **Compute:** is there a GPU machine, or is it Colab/Kaggle? (The models are small, so free Colab is enough.)
6. **Microphone:** budget approval for an I²S MEMS microphone (INMP441 or similar). The Nexys A7 has an onboard PDM microphone, but the ZCU104 has none, so an external I²S microphone on a Pmod is the one that works on both boards. Check the Pmod voltage in the ZCU104 board user guide (UG1267) before wiring anything.
7. **University calendar:** the dates of the mid-term evaluation, any progress report or poster, and the final report and defence. Can a revised synopsis be submitted formally, or do the changes from Step 0.2 go into the mid-term report? Put the dates in the tracker and line Milestone A up with the mid-term evaluation.
8. **ZCU104 tooling:** does the Vivado licence on the lab machines cover the ZU7EV device? (The kit is sold with a Vivado Design Edition seat, but confirm that your lab's copy came with it.) Is the board's power supply and USB cable present? Has anyone in the lab used the ZCU104 with PYNQ or Vitis?

Output. `docs/resources.md`, with every question answered and the date and source of each answer.

Review gate.

☐ Every one of the eight questions has an answer, or an explicit "unknown, asking by DATE."
☐ Both FPGA part numbers are copied exactly as Vivado expects them.
☐ The Cadence tool list says which tools are confirmed and which are assumed.

If it fails. Missing answers don't block Stage 1: it runs on the Nexys A7 or in pure simulation. A missing ZCU104 date or licence blocks only Step 1.11 and the final platform, so keep chasing. Everything still works on the Nexys alone, with the ZCU104 results as an extension.

LLM help. None needed.

Step 0.2 — Rewrite the objectives with no claims

Week 1 · 2 days · Needs: nothing

Goal. Objectives your supervisor signs off that promise only what you can guarantee.

Do. Replace Chapter 4 with something like the following. Adjust the wording, but keep the target/hypothesis split.

> **Design targets**
>
> 1. Assemble a five-class dataset (four firearm classes plus a background/reject class) from public sources, with train/test splits that share no recording session.
> 2. Train and compare a raw-waveform 1D-CNN and a spectrogram 2D-CNN within a fixed hardware budget (≤ 4 M MACs, and few enough parameters to fit the FPGA's distributed RAM, provisionally ≤ 30 k), and report accuracy on the session-disjoint test split.
> 3. Quantize the selected model to 8-bit integers (and explore 4–6 bits), and define an integer-only reference implementation.
> 4. Design a digital CIM macro emulator (weight-stationary, bit-serial, no hard multipliers in the tile array) delivering at least 8 MACs per clock cycle at a target clock of at least 50 MHz.
> 5. Achieve bit-exact agreement between the hardware and the integer reference on the full test set.
> 6. Implement the engine in hand-written RTL and in HLS, and compare resources, clock frequency, latency, initiation interval and design effort.
> 7. Demonstrate live classification from a microphone on an FPGA board.
>
> **Hypothesis to be tested** 8. Because each row of a weight-stationary CIM macro has private weight storage, its inference latency should keep improving as parallelism is added, while a conventional engine fetching weights from shared memory should saturate at its memory-port bandwidth. We will measure inference latency and initiation interval for both architectures across 8–128 parallel MAC units and report the observed scaling.

Also make these edits:

- **Abstract.** Remove the "5×" figure. Say what you will *measure and compare*.
- **Objective 8 (power and energy).** Drop it, or demote it to "vendor-estimated power reported for context."
- **SDG 7.** Soften. You are not measuring energy, so say the architecture *removes repeated weight fetches*, which is an energy-relevant property, without claiming a saving.
- **Accuracy target.** Don't promise 90% before you have seen Sir's dataset. As a reference point, the public Kabealo gunshot dataset's own technical validation (a spectrogram CNN) averaged about 89% across 100 trained models, and a session-disjoint split is a harder test than most published numbers. Use "report accuracy on a session-disjoint split" now, and agree a number with Sir after the Step 0.4 audit.
- **Methodology.** Add one paragraph: "On an FPGA, weights cannot be stored inside a multiplier-capable SRAM array. We implement a CIM macro *emulator*: weights held in distributed-RAM primitives fused to the arithmetic that consumes them, with the dataflow of an all-digital SRAM CIM macro. The tile array (weight storage, AND mask, adder trees, shift-accumulate) uses no DSP slices. Requantization between layers multiplies by a per-channel scale and is reported separately."
- **Scope of the "no DSP" statement.** It applies to the **tile array**, not to the whole engine. Requantization multiplies a 32-bit accumulator by a 16-bit scale, which Vivado places in a DSP slice by default, and it sits outside the macro. Write the statement exactly that way, report DSP use per module, and give the whole-engine total next to it. The baseline

engine uses the identical requantization unit, so the comparison stays fair. Optionally, add one variant with the requantization multiplier forced into logic, to show a fully DSP-free engine; it costs LUTs, so report it as a separate row. Record the final scope as Decision D17 and agree it with Sir.

- **Figures 5.1 and 5.2.** The current block diagram and flowchart are out of date: they feed the spectrogram path straight into live hardware, show no baseline, and show no microphone-to-board path. Note them for redrawing in Steps 3.1 and 4.4, and use the redrawn versions at the mid-term evaluation.
- **Technology target (Chapter 5.1) and costing (Chapter 8).** Name the Nexys A7-100T as the development board and the ZCU104 as the final platform, and describe the design as board-independent. If the boards are lab-provided, the board line in the costing changes; confirm with Sir.

Output. Revised Chapter 4, Abstract, SDG section, Chapter 5.1 and Chapter 8.

Review gate.

☐ Search the document for "×", "times", "reduction", "faster" and "lower". No comparative claim survives outside the hypothesis.
☐ Every design target is either met or not met (nothing vague like "efficient").
☐ Every "no DSP" statement says "tile array" and not "engine" or "design".
☐ Sir has seen and approved it.

LLM help. Opus drafts. Then paste the result into GPT with: "*Find every sentence that claims a result we have not yet measured.*"

Step 0.3 — Repository, tools, build skeleton

Week 1–2 · 3 days · Needs: 0.1 (part number)

Goal. Every team member can clone, simulate and synthesize "hello world" from the command line.

Do.

1. Create a private GitHub repo with this layout:

```
sw/  tb/  hls/  host/  results/  docs/
rtl/core/            board-agnostic design
rtl/board/nexys/     Nexys A7 wrapper (clock, pins, UART)
rtl/board/zcu104/    ZCU104 wrapper
fpga/nexys/  fpga/zcu104/    constraints, TCL, reports
docs/diagrams/       .drawio sources plus exported .svg
```

2. Set up the Python environment: `requirements.txt` with numpy, tensorflow, scikit-learn, librosa, soundfile, matplotlib, pandas, cocotb, pyserial.
3. Install the simulator: Icarus Verilog or Verilator, plus cocotb and GTKWave. (Use Xcelium later if Cadence is confirmed.)
4. Install Vivado ML Standard (free; it includes Vitis HLS) on at least two machines. During installation, select device support for both **Artix-7** and **Zynq UltraScale+ MPSoC**. Confirm in the tool that the ZU7EV part is available to you; if it needs a licence, see Step 0.1 question 8.
5. Write `fpga/tcl/build.tcl`. It takes a top module, a board name and the matching part number, then runs synthesis → implementation → bitstream, and writes the utilization and timing reports to `fpga/reports/<config>/`. No GUI.
6. Write a `Makefile` with targets `sim`, `synth` and `bit`, each taking `BOARD=nexys` or `BOARD=zcu104`.
7. Create `results/runs.csv` with the header:
   `date,commit,board,stage,config,n_macs,f_clk_mhz,cycles,ii,lut,ff,bram,dsp,dsp_array,acc,bitexact_pct,notes` (`dsp` is the whole-design total; `dsp_array` is the DSP count inside the CIM tile array and must be 0 for CIM configurations.)
8. Agree these team rules and write them in `docs/rules.md`:
   - every measured number goes into `runs.csv` with its commit hash;
   - nothing in the report is typed by hand;
   - no LLM ever writes a number into the results.

Output. The repo with a blinking-LED top module that builds to a bitstream with `make bit` and passes a cocotb test with `make sim`.

Review gate.

☐ A *different* member clones it fresh and runs `make sim` and `make bit` successfully.

☐ The blinking-LED bitstream runs on the Nexys A7 (and on the ZCU104 as soon as you have it).
☐ `runs.csv` exists, with one row from the LED build, including its `board`.

LLM help. Sonnet writes the TCL, the Makefile and the cocotb skeleton. Verify TCL commands against the Vivado TCL reference (UG835). Models invent TCL options.

Step 0.4 — Get and audit the dataset

Week 1–2 · 3 days · Needs: nothing · Lane: can run alongside 0.3

Goal. Find out *now* whether Sir's dataset can support five honest classes. Don't train anything yet; this step is just questions and counts.

Do.

1. **Ask Sir these questions about the dataset when he hands it over**, and write the answers in `docs/dataset_audit.md`. The labels alone won't tell you most of this.
   - **Classes:** which weapons, and is there a non-gunshot class?
   - **Collection:** how many separate recording sessions (days, locations, shooters)? How was each one recorded?
   - **Devices:** which microphones or recorders? Was the same shot captured by several devices at once?
   - **Format:** sample rate, bit depth, channels, file format, and whether it was compressed (MP3 or similar) at any point.
   - **Labels:** who labelled it, and are the shot times marked?
   - **Sources:** was any of it taken from the internet, films or games? Those clips need to be kept apart; they can't be treated as field recordings.
   - **Use:** can results, figures and short audio examples go in the report, the slides and any paper? Can the data leave the lab or go onto Colab or Drive?
   - **When:** if it isn't ready yet, on what date will it be?
2. **Build the pipeline so it doesn't care which dataset it reads.** Everything downstream reads `manifest.csv`, never raw folder names. Swapping datasets then means writing one new loader.
3. If Sir's data won't arrive by the end of Week 2, **build and test the pipeline on the Kabealo dataset in the meantime** (Zenodo DOI 10.5281/zenodo.7004819, CC-BY-4.0, 2148 recordings of four firearms). Stage 2 then isn't blocked; you swap in Sir's data when it arrives.
4. Download UrbanSound8K and ESC-50 (hard negatives for the background class) and Google Speech Commands (fallback workload). Download Kabealo anyway: it is a ready-made **cross-dataset test**. A model trained on Sir's data and tested on a different dataset's recordings of a matching weapon type is a strong check on whether it learned the gunshot or the recording setup.
5. Write `sw/data/audit.py`. It produces `manifest.csv` with these columns: `path, class, session_id, device, sample_rate, duration, source`. Define `session_id` as the **recording session (firing sequence)**, not the device. If one shot was captured by several devices at once, every one of those recordings shares the same `session_id`; otherwise the same gunshot lands in both train and test. Keep the `device` column too: holding out a device is a second, harder test you can report.
6. Print, per class: the number of clips, the number of distinct sessions, the number of distinct devices, the sample rates and the sources.

Output. `manifest.csv`, plus `docs/dataset_audit.md` containing Sir's answers and the counts table.

Review gate.

☐ Every question in item 1 has an answer, or "unknown, asked on DATE."
☐ Every firearm class has **at least 3 distinct sessions**. Otherwise it cannot be split honestly; flag it immediately.
☐ The reviewer opens 5 random clips per class and confirms they actually contain a gunshot of that class.
☐ Permission to publish results and to use cloud tools is confirmed in writing.

If it fails. If session information doesn't exist, ask Sir how the recordings were made and reconstruct sessions from file dates, folder structure or recording notes. If a class has fewer than 3 sessions, merge classes (for example handgun / long gun / shotgun / background) or ask Sir whether more recordings exist. Decide this with Sir before Stage 2.

LLM help. Sonnet writes the audit script and the loaders. Gemini reads any documentation that comes with the dataset in one go and lists how sessions, devices and labels are recorded. **If Sir says the data must not leave the lab, don't paste clips or file listings into any LLM.**

STAGE 1 — Real engine, first layer, tested part by part (Weeks 1–13) ← START

HERE

This stage builds the first layer of your real 1D-CNN and runs it on the Nexys A7. There is no separate practice network: everything built here is part of the final engine.

The first layer. A convolution with a 64-sample window, a stride of 8 and 16 output channels, over the 8,000-sample input. That gives 993 output positions × 16 channels. Each output is a 64-input dot product, which one 16×16 tile computes in 4 chunks of 16 inputs. The tile's shape matches the layer, so one tile is enough for this stage.

A quick real model is enough. Bit-exactness doesn't depend on accuracy: the hardware is correct if it produces exactly the same integers as `ref_int.py`, whatever the weights are. But the weights should be realistic, so the quantization scales and value ranges look like the final model's. Stage 1 therefore uses the real network's shape, trained briefly on real clips from Sir's dataset, so the hardware never waits for the full training chain (audit, splits, ADC sweep, selection, quantization). The final trained weights arrive from Stage 2 and are tested the same way (Step 2.8). Its accuracy is never reported.

Random vectors still have a job. The tile and block unit tests keep using random and extreme values (the all −128 case, the single spike, random weights loaded into the tile), because real audio never reaches the corners where bit-serial bugs live.

Test each part as soon as it is ready. Every block gets its own test the day it is finished, before it is connected to anything. As soon as a path to the board exists (Step 1.7), each new piece is tested on the board too. Small tests on small pieces are how a bug takes an hour to find instead of a week.

Step 1.7 (the board link) runs alongside Steps 1.6 and 1.8, so the board is ready the day layer 1 passes simulation.

Step 1.1 — A real-shaped model, quickly trained on real clips

Week 2–3 · 2.5 days · Needs: 0.3, 0.4

Goal. A model with the exact shape of the starting 1D-CNN (Step 2.4) and realistic weights, so hardware work can start before the full training chain finishes.

Do.

1. Define the network once, in `sw/models/cnn1d.py`, in TensorFlow/Keras. Stage 2 trains this same definition; there must never be two versions of the architecture.
2. A script prints each layer's output shape, MACs and parameter count.
3. Cut fixed 0.5 s windows around the labelled shots, using the dataset's own timestamps (the event detector is not needed yet). Background comes from the quiet stretches of the same recordings. Resample to 16 kHz mono and normalize.
4. Train for a few epochs on a laptop CPU (Adam, a plain random 80/20 split) and save `model_quick.keras`. The model has about 15 thousand weights, so this should take minutes; confirm that on the first run.
5. Sanity check only: the model must beat chance and its loss must fall. **Never quote its accuracy.** A random split leaks session information and flatters the number.

What it skips, because those decide the final model, not the hardware test: grouped splits and cross-validation, the ADC sweep, augmentation, several seeds or architectures, the leakage check and the selection rule.

Output. `cnn1d.py`, the per-layer table, and `model_quick.keras`.

Review gate.

- ☐ The shape matches the starting design in Step 2.4, layer by layer.
- ☐ MACs and parameters come from the script, not from hand estimates.
- ☐ Stage 2's training code imports `cnn1d.py` rather than defining its own copy.
- ☐ The quick model beats chance and its loss fell; its accuracy is not quoted anywhere.

If it fails. If Monday's audit shows no usable shot labels or only a few clips, use a model with random weights for the first two weeks (same shape, fixed seed) and record the decision as D18. Nothing downstream changes: only the contents of the weight files do.

LLM help. Sonnet.

Step 1.2 — Write the arithmetic spec for the whole network

Week 1–2 · 5 days · Needs: nothing — *the most important step in Stage 1*

Goal. A single document that defines every integer operation exactly, so that Python and RTL can be written independently and agree.

Do. Write `docs/arith_spec.md`. It must define:

1. **Number formats.** Activations are signed int8 [−128, 127]. Weights are signed int8. Biases are int32. Accumulators are int32.
2. **Quantization.** Symmetric quantization, per-channel for weights and per-tensor for activations, `x_q = clamp(round(x / s), −128, 127)`, with the rounding mode written out explicitly.
3. **Requantization** from an int32 accumulator to an int8 activation:

   ```
   y = clamp( (acc * M0 + (1 << (n − 1))) >> n , lo, hi )
   ```

   - `M0` is a fixed-width integer (pick 16 bits) and `n` is a shift.
   - **State the width of the product `acc * M0`.** A 32-bit signed accumulator times a 16-bit multiplier needs up to 48 bits. Write it in the spec as a 48-bit signed intermediate (64-bit in C and Python), so the Python reference, the C code and the RTL cannot disagree about overflow.
   - The `>>` is an arithmetic shift. State what happens for negative values.
   - ReLU is folded in as `lo = 0`.
4. **Bit-serial dot product.** Activation bit-planes are processed `b = 0…7`. Partial sum at bit `b` = the sum of the weights whose activation bit is 1. The accumulator adds `psum << b` for `b < 7` and **subtracts** `psum << 7` for `b = 7`, because the top bit of a two's-complement number has weight −128.
5. **Layer types.** Convolution as a dot product: for output position `p` and channel `c`, which input samples it reads (start index, stride, padding) and in what order they are split into 16-input chunks. Max pooling on int8 values. Global average pooling as a sum, with the `1/L` folded into the next layer's requantization so no integer division happens in hardware. Dense layers. ReLU (folded into requantization as `lo = 0`) and argmax. Cover every layer type in the starting 1D-CNN now, so the spec doesn't need rewriting in Stage 3.
6. **Accumulator bound.** Prove that int32 cannot overflow for the largest layer: `max |acc| ≤ inputs × 128 × 128 + |bias|`. Write the number down.
7. **Worked example.** Hand-compute one 4-input dot product bit-serially. Include a −128 activation and a negative weight. Show every cycle.
8. **Draw the verification ladder** as `docs/diagrams/verification_ladder.drawio`: float model → integer reference → Python tile model → RTL simulation → post-synthesis → post-implementation → board, with the pass criterion written on each rung. It is half an hour's work and it is the picture you will put in the report's methodology chapter and show at every evaluation.

Output. `docs/arith_spec.md` and `docs/diagrams/verification_ladder.{drawio,svg}`.

Review gate.

☐ **Two members independently hand-compute the worked example** from the spec, without looking at each other's work. Both get the same answer, and it equals the ordinary `sum(w*a)`.
☐ Every layer type in the starting 1D-CNN is defined, including exactly where each window starts and ends.
☐ Every operation has its rounding, shift and clamp behaviour spelled out. The reviewer tries to find any operation where two reasonable people could implement it differently. Every one found gets fixed.

If it fails. If the hand computations disagree, the spec is ambiguous. That is exactly what this step exists to catch. Fix the spec, never the arithmetic.

LLM help. Opus helps draft and checks the overflow bound. Then a *different* model (GPT) independently hand-computes the worked example from the spec. If it disagrees with yours, find out why.

Step 1.3 — Integer reference for the whole network

Week 3–4 · 5 days · Needs: 1.1, 1.2

Goal. `ref_int.py`: a pure-integer forward pass for every layer of the real network, implementing `arith_spec.md` exactly.

Do.

1. `sw/quant/quantize.py`: quantize `model_quick.keras` following the spec. Calibrate the activation scales on real clips from the quick model's training windows. Produce `model_int.npz` with int8 per-channel weights, int32 biases and `(M0, n)` per output channel. The same script quantizes the trained model in Step 2.6.
   - **Make it check its own output.** After producing the artifact, `quantize.py` asserts that every multiplier and shift is within the range `arith_spec.md` allows, and computes the worst-case accumulator for each output channel from the actual weights (sum of absolute weights × 128, plus the bias) and asserts it stays under the int32 bound. If an assertion fails, the model and the spec disagree, and you know before touching hardware.
2. `sw/ref/ref_int.py`: a forward pass for every layer type in the spec, using only numpy `int32`/`int64` arrays. `assert` the dtype at every step. A function `trace(x)` returns every intermediate tensor.
3. **A second, independent implementation of layer 1.** A team member who did not write `ref_int.py` writes a plain nested-loop version of the first layer from the spec alone. The two must agree exactly.

Output. `quantize.py`, `model_int.npz`, `ref_int.py`, and the independent layer-1 version.

Review gate.

☐ No floats anywhere in the forward pass (the dtype asserts pass).
☐ The independent layer-1 version matches `ref_int.py` exactly on 100 random inputs.
☐ The reviewer hand-computes one layer-1 output (one position, one channel) and it matches `trace()`.
☐ Every multiplier and shift is within the spec's range, and the worst-case accumulator of every output channel is below the int32 bound (asserted by `quantize.py`).

The quick model's accuracy is never reported. Real accuracy is measured in Step 2.7, with the trained model.

If it fails. Find the first position and channel where the two implementations differ and check both against the spec. The usual causes are the window start index, the rounding in requantization, and the order of the per-channel scales.

LLM help. Opus for `ref_int.py`. The independent layer-1 version must come from a different person and a different model, working only from the spec.

Step 1.4 — Python bit-serial tile model

Week 3 · 2 days · Needs: 1.2

Goal. A cycle-level Python model of the hardware tile. It is the bridge between the "math" reference and the RTL.

Do. `sw/ref/tile_model.py`:

1. Implement the 16×16 tile exactly as the spec says: 8 cycles, with the bit-plane broadcast, the column sums and the MSB subtract.
2. Test it against `numpy` `W.T @ a` on 100,000 random vectors, plus edge cases:
   - all activations −128;
   - all activations +127;
   - all weights −128;
   - all zeros.

Review gate.

☐ 100% match on random and edge cases.
☐ The reviewer reads the code against `arith_spec.md` section 4, line by line.

LLM help. Sonnet writes it. A different model writes the edge-case list.

Step 1.5 — Test vector generator and file formats

Week 4 · 2 days · Needs: 1.3, 1.4

Goal. A fixed, documented format for everything Python hands to RTL.

Do. `sw/vectors/emit.py` produces:

- `tile_rand.hex`: random tile tests (weights, activations, expected outputs);
- `weights_l1.mem`: layer-1 weights in `$readmemh` format, laid out by tile slot, with the layout documented;
- `inputs.hex`: 20 8,000-sample inputs: 10 real clips, 6 random inputs of mixed amplitude, and 4 extreme vectors (all −128, all 127, all 0, a single spike);
- `expected_l1.hex`: the expected layer-1 output for each input, every position and channel.

Document the byte order, the bit order and the line format in `docs/file_formats.md`. Write the generator so it can emit every layer later, not only layer 1.

Review gate.

☐ The reviewer writes a 10-line reader that parses each file back and recovers the original arrays.

LLM help. Sonnet.

Step 1.6 — CIM tile RTL and its testbench

Week 4–5 · 2 weeks · Needs: 1.4, 1.5

Goal. A synthesizable, verified `cim_tile`. This module is the core of the whole project.

Do.

1. Write `rtl/cim/cim_tile.sv`:
   - parameters `R=16, C=16, WBITS=8, ABITS=8, DEPTH`;
   - weights in a `DEPTH × (C·WBITS)` distributed-RAM array per row, with a write port for preload and a read port addressed by `slot`;
   - per cycle, the inputs are `a_bits[R]` (one bit per row) and a `bit_idx`, and it performs the AND-mask, the adder tree per column and the shift-accumulate;
   - put `(* use_dsp = "no" *)` on the tile and on the adder tree. Requantization is **not** part of the tile; it lives in `rtl/cim/requant.sv` outside the macro, and may use DSP slices.
2. Write `rtl/common/adder_tree.sv`, parameterized, with an optional pipeline register stage.
3. **Draw the tile datapath and the bit-serial timing diagram before writing the RTL**, from `arith_spec.md` only: `docs/diagrams/tile_datapath.drawio` and `docs/diagrams/tile_timing` (WaveDrom). Put the bit width on every wire, mark every pipeline register, and show the MSB subtract explicitly. Drawing this first is how you find out whether the spec is actually implementable; redraw the clean version once the testbench passes.
4. Write the cocotb testbench `tb/test_cim_tile.py`. It reads `tile_rand.hex` and runs:
   - 10,000 random tests;
   - directed tests for every edge case from 1.4;
   - a preload/reload test (write slot 0, compute, write slot 1, compute, return to slot 0 → still correct).
5. Lint: `verilator --lint-only -Wall`.

Output. The RTL, the testbench, the lint log, and the datapath and timing diagrams.

Review gate.

☐ 100% pass, random and directed.
☐ The lint log is clean: no latches, no width warnings left unexplained.
☐ The reviewer deliberately breaks the MSB subtract (changes it to an add) and confirms the testbench *fails*. A testbench that cannot catch the most likely bug is not a testbench.
☐ A reviewer who did not draw it checks the datapath diagram against the RTL: every block is a module, every module is a block, and every bit width matches.

If it fails. Compare cycle by cycle against `tile_model.py`. Add a debug dump of `psum` at each bit position from both.

LLM help. Sonnet writes the RTL and the testbench, but in *separate conversations*, each from `arith_spec.md` only. If a bug survives an hour, take it to Opus with the spec, the RTL and the failing vector.

Step 1.7 — Board bring-up and the UART link

Week 4–6 · 5 days · Needs: 0.3, and 1.6 for the last part · Lane: runs alongside Steps 1.6 and 1.8

Goal. A tested path between the PC and the board before there is an engine to put on it, so layer 1 goes onto the board the day it passes simulation.

Do this in strict order. Each part works before the next starts.

1. **Blinking LED** on the Nexys A7 (from Step 0.3).
2. **UART loopback.** The board echoes bytes. A host script sends 1,000 random bytes and checks the echo.

3. **UART protocol.** Define and document it in `docs/uart_protocol.md`: a start byte, a command byte, a length, the payload and a checksum. You need commands to load an input (8,000 bytes), start the engine, and read back the result and the cycle count (uint32). At 115,200 baud, one input plus the 993 × 16 layer output takes a couple of seconds; use a higher baud rate if both ends support it.
4. **The tile alone on the board.** Once Step 1.6 passes, put the tile behind the UART. The host sends weights and activation vectors, the board returns the column sums, and `host/run_tile.py` compares them with `tile_model.py`.

Output. The UART RTL, `docs/uart_protocol.md`, `host/run_tile.py`, and the results row in `runs.csv`.

Review gate.

- ☐ The loopback test passes with 1,000 random bytes.
- ☐ A non-author has reviewed the protocol document.
- ☐ The tile on the board matches `tile_model.py` on 10,000 random vectors.
- ☐ The reviewer reproduces the tile test on their own laptop.

LLM help. Sonnet for the UART RTL and host scripts. Write the UART receiver from a known-good reference design rather than letting a model invent one.

Step 1.8 — Layer-1 engine, block by block

Week 5–7 · 3 weeks · Needs: 1.5, 1.6

Goal. The real first layer running on the tile in simulation, bit-exact, with every block tested before it is connected.

Diagram first. Draw `docs/diagrams/layer1_engine.drawio` (tile, input and output buffers, window addressing, requantization, data widths) and the controller state machine with the cycles per state. The state machine is where the cycle-count formula comes from.

Do. Build each block and test it alone, against a Python reference, the day it is finished. For `requant_pc`, draw the multiplier, shift and bias at random across their full legal ranges as well as the accumulator, so no model can hand the hardware a combination the test hasn't seen:

Block	What it does
`act_buffer`	Holds the 8,000-sample input and the layer output
`im2col_addr`	For output position `p`, generates the addresses of samples `8p` to `8p + 63`, as 4 chunks of 16
`requant_pc`	Per-channel requantization and ReLU (outside the macro; the expected DSP user)
`perf_counters`	Cycles, weight writes, weight reads

Then assemble them with one `cim_tile` (layer-1 weights preloaded from `weights_l1.mem`) and a controller that loops over 993 positions × 4 chunks × 8 bit-cycles. Run every input in `inputs.hex` and compare every position and channel with `expected_l1.hex`.

Keep the layer's shape as parameters: window, stride, channel count and input length. The model may change after Stage 2, and the same blocks are reused for the later layers in Stage 3.

Output. The blocks and their unit tests, the assembled layer-1 engine and its testbench, and the diagrams.

Review gate.

- ☐ Every block's unit test passes, and the reviewer broke one thing in each block and the test caught it.
- ☐ The assembled layer is bit-exact on every position and channel, for every input.
- ☐ The cycle count matches the reviewer's hand calculation from the state-machine diagram.
- ☐ The weight-write counter stays 0 during inference.
- ☐ A non-author checks the block diagram and state machine against the RTL.

If it fails. If the unit tests pass but the assembled layer doesn't, check the window start (`8p`, not `8p + 1`), the last position (index 992, the 993rd), and the order of the per-channel scales.

LLM help. Sonnet for each block and its test. Opus for any "mostly matches" bug: 99% agreement nearly always means a rounding, sign or off-by-one error.

Step 1.9 — Synthesize the layer-1 engine

Week 7 · 3 days · Needs: 1.8, 0.3

Do.

1. Run `make synth TOP=layer1_engine BOARD=nexys` at 100 MHz.
2. Read the **hierarchical** utilization report (LUT, FF, LUTRAM, BRAM, DSP per module; `report_utilization -hierarchical`, confirm the option in UG835) and the timing report (WNS). The per-module view is how you separate the tile array from requantization.
3. Run a post-synthesis functional simulation on 5 inputs.

Review gate.

☐ **DSP = 0 inside the tile array** in the hierarchical report. Any DSPs in the design belong to requantization and are reported as such, with the whole-design total.
☐ WNS ≥ 0 at 100 MHz, or you have recorded the achieved frequency and the critical path.
☐ The post-synthesis simulation is bit-exact.
☐ A row is added to `runs.csv`.

If it fails. If WNS < 0, add an adder-tree pipeline stage (a parameter change). If DSPs appear inside the tile array, check the `use_dsp` attribute placement. DSPs in the requantization unit are expected.

LLM help. Gemini reads the full synthesis and timing reports and identifies the critical path and any inferred DSP.

Step 1.10 — Layer 1 on the Nexys A7

Week 8 · 4 days · Needs: 1.7, 1.9

Goal. The first hardware result for the real network. The UART link and the tile were already tested on the board in Step 1.7, so this step only adds the engine.

Do.

1. Put the layer-1 engine behind the UART link from Step 1.7.
2. `host/run_layer1.py` sends every input from `inputs.hex`, reads back the 993 × 16 output and the cycle count, and compares both with `ref_int.py`.

Output. `run_layer1.py` and the results row in `runs.csv`.

Review gate.

☐ Every position and channel matches the reference for every input, on the Nexys A7.
☐ The board cycle count equals the simulation cycle count.
☐ The reviewer reproduces the run on their own laptop.

🏁 **Milestone A — "The first real layer works on the board."** Demo this to Sir. The chain from model to board now exists for the real network, and the remaining layers are added one at a time in Stage 3. This is a good result to show at the mid-term evaluation.

LLM help. Sonnet for the host script.

Step 1.11 — Layer 1 on the ZCU104

Week 12–13 · 4 days · Needs: 1.10, and the ZCU104 in hand

Goal. Find every ZCU104 surprise (licence, clocking, pins, host link) while the design is still small, instead of in Week 14 with the full engine.

Do.

1. Create `rtl/board/zcu104/` and the ZCU104 constraints. **Take pin names and locations from AMD's master constraints file for the board, never from an LLM.** A wrong pin assignment can look fine and drive the wrong thing.
2. Get the blinking-LED build running on the ZCU104 first.
3. Choose how the PC talks to the fabric. Try them in this order:
   - **Option A, fabric only.** Keep the same UART block and protocol as the Nexys, with an external 3.3 V USB-UART module on a Pmod connector (your costing already includes a USB-UART module). The design is programmed over JTAG. **Test on day one that this works on your board setup**, because the Zynq's processor system normally

has to be considered even for fabric-only designs. The clock comes from the board's fabric clock; check the user guide for the pin and frequency.
    - **Option B, through the processor system.** Build a Zynq UltraScale+ block design: the fabric clock from the processor system, a small AXI-Lite register block (start, status, cycle counter, weight-write counter, input and output buffers), and a host program on the board's own USB-UART, either bare-metal in Vitis (written in C) or Python under PYNQ. Use Option A if it works. Take Option B if A is awkward, or if you intend the DRAM-bound stretch, which needs the processor system anyway. Record the choice as Decision D15.
4. Run the layer-1 test inputs through the layer-1 engine on the ZCU104.

Review gate.

☐ Every layer-1 output matches `ref_int.py`, the same result as Step 1.10.
☐ The cycle count equals the Nexys cycle count exactly. Cycles come from the design, not the board; a difference means a wrapper bug.
☐ `make bit BOARD=zcu104` builds from a clean clone, and its row is in `runs.csv` with `board = zcu104`.
☐ Decisions D14 (measurement platform) and D15 (host interface) are recorded.

If it fails. If the ZCU104 isn't available yet, mark this step `Cut` in the tracker and revisit it when the board arrives. Nothing downstream is blocked.

LLM help. Sonnet writes the wrapper and TCL; verify block-design and constraint details against the ZCU104 user guide (UG1267) and the Vivado documentation. Opus if the PS–PL clocking or reset behaves strangely.

STAGE 2 — Real data and real model (Weeks 3–11)

Lane. This stage runs alongside Stage 1. It needs Step 0.4, the network definition from Step 1.1, and the arithmetic spec and integer reference from Steps 1.2 and 1.3 by the time it reaches quantization (2.6).

Step 2.1 — Preprocessing and session-disjoint split

Week 3–4 · 2 weeks · Needs: 0.4

Do.

1. **Write the event detector the hardware will use**, in Python: energy threshold, refractory period, integer arithmetic only. Save it as `sw/frontend/detector.py`. The RTL detector in Step 4.3 must match it exactly.
2. **Cut every training window with that detector**, not with the dataset's timestamps: 0.5 s, starting a little before the trigger. Use the timestamps only to check that the detector found the right shots. If training windows are aligned differently from the windows the hardware cuts, live accuracy drops for reasons that have nothing to do with the model.
3. Resample to 16 kHz, convert to mono and normalize.
4. **Build the background class mostly from the same recordings**: the stretches between shots. Add hard negatives (claps, door slams, fireworks and other sharp sounds) from UrbanSound8K and ESC-50, passed through the same resampling. If gunshots come from one dataset and background from another, the model can learn to recognise each dataset's microphone instead of the gunshot.
5. **Split by session, using cross-validation.** With only a few sessions per class, one 70/15/15 split gives a noisy accuracy that depends on which sessions landed in the test set. Instead:
    - hold out one group of sessions as the final test set, used once at the very end;
    - on the remaining sessions, use grouped k-fold cross-validation (scikit-learn `GroupKFold`, with `session_id` as the group) for all training and model selection;
    - report the mean and spread across folds. Save the folds and the test set in `splits.json`.
6. *Also* make a random (clip-level) split. You will report both, to show how much session leakage inflates accuracy.

Review gate.

☐ A script proves that no `session_id` appears in more than one fold, or in both the folds and the test set.
☐ Every training window was cut by `detector.py`. The detector finds at least 95% of the timestamped shots, or its miss rate is recorded.
☐ At least half of the background class comes from the same recordings as the gunshots.
☐ The reviewer listens to 10 extracted windows and confirms the shot is inside each one.

Step 2.2 — Fallback workload

Week 7 · 2 days · Needs: 0.4 (needed before the Week 8 decision, not sooner)

Do. Write a loader for Speech Commands (10 keywords + silence/unknown), producing the same 0.5 s, 16 kHz shape.

Review gate.

☐ The same training script runs on it unchanged.

Step 2.3 — ADC sweep

Week 5 · 1 week · Needs: 2.1

Do.

1. Resample to 8 / 16 / 22.05 kHz and requantize to 16 / 12 / 10 / 8 / 6 bits.
2. Train a small fixed CNN at each point, with 3 seeds.
3. Plot the accuracy heatmap.
4. At the chosen rate and bit depth, compare 0.25 s and 0.5 s windows. The 1D-CNN's MACs grow in proportion to the window length, and the window is also the wait before an inference can start.

Output. The heatmap and the chosen (rate, bits, window length).

Review gate.

☐ The chosen point is the cheapest one within 1 percentage point of the best, and that rule was written down *before* looking at the results.

Step 2.4 — Float models within the hardware budget

Week 5–7 · 3 weeks · Needs: 2.1, 2.3

Which network are we using? Decided now so nobody is stuck:

- **Hardware model: a small 1D-CNN on the raw waveform.** It needs no FFT hardware: microphone → engine directly. Its large first layer (a learned filterbank) is also the layer where activation broadcast helps most.
- **Software comparison: a 2D-CNN on a log-mel spectrogram, plus MFCC + SVM, plus a depthwise-separable CNN.** These answer the research question "waveform vs spectrogram" in accuracy and MACs.
- **Promotion rule (agree it now, before training):** the 2D-CNN only goes into hardware if it beats the 1D-CNN by more than an agreed margin (suggestion: 5 points) on the session-disjoint test. Even then it gets synthesized and simulated, not demoed live.

Budget, for every model: ≤ 4 M MACs per inference. Parameters are limited by where the CIM weights live, which is distributed RAM (LUTRAM). An XC7A35T has about 400 kbit of it in total and an XC7A100T about 1.2 Mbit (check the Artix-7 datasheet, DS180, for your part), and the same slices are also needed for logic. **Provisional limit: 30 k parameters (240 kbit at 8 bits)**, to be confirmed in Step 3.1. **The limit is set by the Nexys A7-100T**, the smaller board, so the same model runs on both. The ZCU104 has several times more memory (check its datasheet), but a bigger model would no longer fit the Nexys and the two boards would stop being comparable. Layers are limited to conv, maxpool, ReLU, global-average-pool and dense. Batchnorm is allowed during training but gets folded in at export.

Tile-friendly shapes. Make each layer's output channels a multiple of 16 (the tile width) and each layer's `k × Cin` a multiple of 16 (the tile height) wherever you can. Otherwise part of a tile sits idle: 24 channels on 16-wide tiles leaves a quarter of the second tile unused on every cycle.

Starting 1D-CNN (about 2.7 M MACs and 15 k weights, with every layer tile-friendly):

```
Input 8000×1 → Conv1D k64 s8 16ch → MaxPool4 → Conv1D k9 32ch → MaxPool4
→ Conv1D k9 32ch → GlobalAvgPool → Dense 32→5
```

This architecture is defined once, in `sw/models/cnn1d.py` (Step 1.1), and Stage 1 is already building its first layer in hardware. If training leads you to change it, change it there and tell whoever is working on the engine, because the layer parameters in the RTL change with it.

Do.

1. Train each model on every cross-validation fold, with augmentation (gain, noise, time shift, reverberation).
2. Log the fold-mean accuracy and its spread, the random-split accuracy, the MACs and the parameters.

3. Run the **leakage check**: train a small model on the four firearm classes using only the 50 ms *before* each trigger, which contains background noise and no gunshot. Chance is 25%. If it scores well above chance, the classes can be told apart by recording conditions alone, and the main accuracy is inflated by that amount.

Output. Model files, the accuracy-vs-MACs Pareto plot and a comparison table.

Review gate.

- ☐ Every model's MACs are computed by a script, not estimated.
- ☐ Cross-validated accuracy (mean and spread) and random-split accuracy are both reported.
- ☐ The leakage check scored close to chance, or its result is reported next to the main accuracy.
- ☐ The test set was used **once**, at the end. All model selection used validation only.

If it fails. If the session-disjoint accuracy is far below target, this is the decision point. Options: more recordings from Sir, fewer classes, or switching the primary workload to Speech Commands. **Decide with Sir by Week 8.** Don't carry the problem forward.

LLM help. Sonnet for the training code. Opus to review the experimental protocol for leakage. Ask it explicitly: "how could information from the test set have reached training?"

Step 2.5 — Model selection gate

Week 8 · meeting

The team decides the hardware model using the promotion rule from 2.4, and records the decision in `docs/decisions.md`.

Step 2.6 — Quantize the selected model

Week 8–10 · 3 weeks · Needs: 2.5, 1.2 (arith spec)

TensorFlow note. Train in Keras, but do not use the TFLite converter's int8 output as your hardware reference. TFLite quantizes activations asymmetrically with a zero point, and `arith_spec.md` defines symmetric activations with no zero point, which is much cheaper in hardware. Run your own PTQ in numpy from the Keras weights and calibration activations, exactly as the spec says. If PTQ loses too much accuracy, do QAT with `tf.quantization.fake_quant_with_min_max_vars` using symmetric ranges. TFLite is still useful as an accuracy cross-check.

Do.

1. **PTQ** (post-training quantization): per-channel weights, per-tensor activations, percentile calibration.
2. **QAT** (quantization-aware training) only if PTQ loses more than 1%.
3. Bit-width sweep: W8A8, W6A6, W4A8, W4A4.
4. Fold batchnorm into the conv weights and check numerically that folded = unfolded.

Review gate.

- ☐ The W8A8 integer accuracy is within 1 percentage point of float.
- ☐ Batchnorm folding is verified numerically.

LLM help. Opus. The scaling math is where silent errors live.

Step 2.7 — Run the trained model through the integer reference

Week 10 · 3 days · Needs: 2.6, 1.3

Do.

1. Export the trained, quantized model from Step 2.6 in the same format as `model_int.npz`. `ref_int.py` already handles every layer type, from Step 1.3.
2. Run the test set through `ref_int.py` and compare its accuracy with the quantized model's.
3. If the selected model uses anything the spec doesn't cover yet (for example, the 2D-CNN's 2D windows), add it to `arith_spec.md` and `ref_int.py` first, and have two members hand-check one output of the new layer type.

Review gate.

- ☐ `ref_int` accuracy equals the quantized-model accuracy.
- ☐ Any new layer type was added to the spec and hand-checked by two members.

Step 2.8 — Real test vectors

Week 11 · 2 days · Needs: 2.7, 1.5, 1.10

Do. Extend `emit.py` for the trained model: weights for every layer, 200 test clips and per-layer expected tensors. Then re-run the layer-1 board test from Step 1.10 with the trained weights. Only the weights change, so it's a quick check that the trained model behaves like the random one.

Review gate.

☐ The round-trip parser from 1.5 still works on the new files.
☐ The layer-1 engine is still bit-exact on the board with the trained weights.

STAGE 3 — Scale the engine and build the baseline (Weeks 8–15)

Step 3.1 — Mapping spec: how real layers run on tiles

Week 8–9 · 2 weeks · Needs: 1.10 (Milestone A), 2.5

Goal. On paper first: exactly how each layer of the real model is scheduled on `NUM_TILES` tiles.

Do. Write `docs/mapping_spec.md`. It must cover:

1. **Conv1D as dot products.** Each output (position p, channel c) is a dot product over `k × Cin` inputs. These are split into chunks of 16 rows, with partial sums accumulated across chunks. Requantization happens only after the last chunk.
2. **Weight placement.** Which weights go in which tile and which slot. Total slots needed = `ceil(params / 256)`. Check that it fits the LUTRAM capacity of the **Nexys A7-100T** (the smaller part) from the Artix-7 datasheet (DS180).
3. **Activation buffers.** Ping-pong between layers and their sizes. Check that they fit BRAM.
4. **The schedule.** The cycle count per layer as a formula, and the total.
5. **Instrumentation.** Counters for cycles, weight reads (should be 0 during inference) and activation reads.
6. **Requantization units.** How many there are and how many output columns share each (a parameter; start with one per tile). Requantization runs once per output, after all bit-cycles and chunks, so units can be shared, but check in the schedule that the tile array never stalls waiting for one.
7. **Diagrams.** `docs/diagrams/layer_mapping.drawio`: one conv layer split into 16-row chunks, which tile and slot holds which weights, and the ping-pong buffers. Also draft `docs/diagrams/system.drawio` (microphone → I²S receiver → event detector → ring buffer → engine → host link), which replaces synopsis Figure 5.1 and is finalised in Step 4.4.

Review gate.

☐ The reviewer computes the total cycles for the real model from the formula. Write the number down; Step 3.5 will check it.
☐ The LUTRAM and BRAM use fit the part, with a margin of at least 20%.
☐ The layer-mapping diagram is checked against the written schedule by a non-author.
☐ The number of requantization units and their sharing are stated, and the schedule shows no stall waiting for them.

LLM help. Opus. This is a judgment-heavy architecture step.

Step 3.2 — Baseline engine spec and fairness review

Week 9–10 · 2 weeks · Needs: 3.1

Goal. Define the conventional ("von-Neumann") baseline *before* building it. Get it reviewed for fairness while changes are still cheap.

Do. Write `docs/baseline_spec.md`. The baseline must have:

- **the same** MAC count, the same adder trees, the same precision, the same requantization, the same clock and the same model;
- **one difference:** weights live in BRAM, away from the MACs, and a controller fetches them into a small buffer before use. The MACs stall when they are starved;
- a prefetch / double-buffer, so it is a *competent* baseline and not a crippled one;

- **a default weight bandwidth equal to the most the weight BRAMs can deliver**, not one narrow port. Weights filling N BRAM36 blocks can be striped across all of them and read through both ports of each, which gives roughly N × 2 × 72 bits per cycle. A single 64-bit port is the first thing an examiner will call a strawman;
- `WPORT` swept from narrow up to that maximum, so the report shows exactly where the baseline stops being limited by bandwidth.

Also write into the spec, before any results exist: at full bandwidth, the baseline may match CIM's latency at this model size. If it does, the comparison moves to cost (LUTs and FFs per MAC-per-cycle) and to how each design scales. That is still a valid result.

Diagram. Draw `docs/diagrams/cim_vs_baseline.drawio`: both engines side by side, same MAC array, same adder trees, same requantization, with the *only* difference being where weights come from and the stall path that creates. Draw it as part of writing the spec, not afterwards: if the two sides don't look identical except for that one thing, your baseline isn't fair yet. This is the figure your viva turns on.

Review gate (fairness):

☐ A team member who did not write the spec answers in writing: "*If I were the examiner, how would I argue this baseline was made deliberately weak?*" Every point is either fixed or answered in the spec.
☐ Opus and GPT are each given the spec with the same question. Their points are addressed.
☐ The spec answers "why not just keep the weights in registers next to the MACs?" That is what a fully partitioned HLS design does (Step 5.5), so the HLS result becomes a third reference point: compare LUT and FF cost per MAC-per-cycle across CIM, baseline and HLS.
☐ The side-by-side diagram exists, and a non-author confirms the two datapaths differ only in weight supply.
☐ Sir has seen it.

Step 3.3 — Cycle model for both engines

Week 10–11 · 2 weeks · Needs: 3.1, 3.2

Goal. Predict the cycle counts of both engines from the specs, before any RTL exists.

Do.

1. `sw/simcycle/`: a Python cycle model for the CIM engine and the baseline, parameterized by `n_macs` and `WPORT`.
2. Sweep `n_macs ∈ {8, 16, 32, 64, 128}`.

Output. A *predictions* table and plot, saved as `results/predictions_v1.csv`.

Review gate.

☐ The CIM prediction for layer 1 equals the measured layer-1 cycle count from Step 1.10.
☐ Prediction files are labelled "prediction." **None of these numbers goes into any claim.** They exist only to be checked against the hardware in Stage 6.

LLM help. Sonnet writes it. Opus checks the stall model of the baseline.

Step 3.4 — Remaining engine blocks

Week 9–11 · 3 weeks · Needs: 3.1, 1.8

Do. Build and unit-test each remaining block *separately*, the day it is finished. The blocks from Step 1.8 are extended here, not rewritten:

Block	What changes
`act_buffer`	Extended to ping-pong between layers
`im2col_addr`	Extended to multi-channel inputs and padding (layers 2 and 3)
`layer_seq`	New: walks the layer schedule from the mapping spec
`maxpool`	New: int8 max over the pool window
`gap_sum`	New: sum for global average pooling
`argmax`	New: picks the class
multi-tile control	New: spreads one layer's work across `NUM_TILES` tiles

Each block gets its own cocotb test, with expected values from the Python reference.

Review gate (per block).

☐ The unit test passes.
☐ Lint is clean.
☐ The reviewer breaks one thing in the block and confirms the test catches it.

Step 3.5 — Integrate the full engine, one layer at a time

Week 11–12 · 2 weeks · Needs: 3.4, 2.8

Do. Add one piece at a time to the working layer-1 engine, in this order, and test each addition in simulation and then on the Nexys before adding the next:

layer 1 (already working) → max pool → layer 2 → max pool → layer 3 → global average pool → dense → argmax → several tiles

Each addition compares every intermediate tensor with `ref_int.py` and logs a row in `runs.csv`. Finish with the 200 real clips and the trained weights.

Review gate.

☐ Every addition passed in simulation and on the board before the next one was added (each has a row in `runs.csv`).
☐ Bit-exact on every layer for all 200 clips.
☐ The cycle count equals the formula from 3.1.
☐ The weight-read counter reads 0 during inference.

If it fails. The bug is almost always in the last addition, which is the point of adding one at a time. The usual causes:

- the order of partial-sum accumulation across chunks;
- per-channel requantization indexing;
- off-by-one errors in `im2col` at layer edges (padding, stride).

Step 3.6 — Baseline engine RTL

Week 11–13 · 3 weeks · Needs: 3.2, 3.4 · Lane: runs alongside 3.5

Do. Reuse the MAC array and all the Step 3.4 blocks. Replace weight storage with a BRAM weight memory plus the fetch controller from the spec.

Review gate.

☐ Bit-exact on all 200 clips.
☐ **The baseline and CIM produce identical outputs on all clips**: the same arithmetic with different data movement. If they differ, one of them is wrong.
☐ The cycle count matches the cycle-model prediction within 10%. If not, explain the difference before moving on.

Step 3.7 — Parameter sweep in simulation

Week 14–15 · 2 weeks · Needs: 3.5, 3.6 · Lane: runs alongside 4.1

Do. Run both engines at each `n_macs` value in simulation and log the cycles. Compare against `predictions_v1.csv`.

Review gate.

☐ Every configuration is still bit-exact.
☐ A table of predicted vs simulated cycles exists, with every difference explained.

STAGE 4 — FPGA implementation and board (Weeks 12–17)

Step 4.1 — Build all configurations

Week 13–14 · 2 weeks · Needs: 3.5, 3.6, 1.11 (synthesis needs bit-exact RTL, not the simulation sweep, so it runs alongside 3.7)

Do.

1. `make synth` for each of CIM and baseline × each `n_macs`, plus the HLS variants (Stage 5) when they are ready, **once per board**. On the Nexys, sweep as far as it fits. On the ZCU104, extend the sweep to larger MAC counts (for example 256 and 512) if they fit and close timing. The clock target may differ per board; use the same target for CIM and baseline on a given board.
2. Harvest the reports into `runs.csv` automatically.

Review gate.

☐ Every configuration has WNS ≥ 0 at its target clock, or a recorded achieved frequency.
☐ Every CIM configuration shows DSP = 0 inside the tile array (hierarchical report). Requantization DSPs are reported separately, and both `dsp` and `dsp_array` are in `runs.csv`.
☐ A post-route timing simulation on 20 clips is bit-exact for at least the largest CIM and baseline configurations.

If it fails. If large configurations don't fit or don't close timing, drop the largest `n_macs` point and report it as "did not fit the part." That is a valid observation, not a failure. Keep the two boards' results in separate tables.

LLM help. Gemini on the full Vivado logs. Sonnet on the harvesting script.

Step 4.2 — Full test set on the boards over UART

Week 14–15 · 2 weeks · Needs: 4.1, 1.10

Do. The UART protocol from Step 1.7 already carries real inputs. Run the full test set through the CIM engine and the baseline, logging the class, the logits and the cycle count.

Review gate.

☐ **100% logit match** with `ref_int.py` on the full test set, for both engines.
☐ Board cycle counts equal the simulation cycle counts.

Step 4.3 — Microphone path

Week 12–14 · 3 weeks · Needs: 0.1 (mic bought) · Lane: runs alongside 4.1–4.2

Which microphone. The Nexys A7 has an onboard PDM microphone (ADMP421). It outputs a 1-bit stream, so using it means building a PDM-to-PCM filter chain first, and none of that carries over to the ZCU104, which has no microphone. **Use an external I²S MEMS microphone on a Pmod**: the same receiver works on both boards. Use the onboard PDM microphone only as a quick sanity check if the I²S microphone hasn't arrived yet. Record the choice as Decision D16.

Do this in strict order.

1. **I²S receiver.** Mic → FPGA → raw samples streamed over UART → host plots the waveform. Check it visually (clap, speech).
2. **Truncation** to the bit depth chosen in 2.3, plus resampling if needed.
3. **Event detector.** Energy threshold, refractory period, ring buffer that captures the 0.5 s window.
4. **Clock-domain crossing** from the I²S clock to the system clock, using a known-good asynchronous FIFO or synchronizer template.

Review gate.

☐ Recorded waveforms match a reference recording of the same sound made on a laptop.
☐ 1,000 synthetic clicks over 10 minutes are all detected with no lockups.
☐ The microphone path works on the board used for the final demo.

LLM help. Sonnet for the host plotter. Do **not** let a model write the CDC logic from scratch; use a proven template and have Opus review it.

Step 4.4 — Live demo integration

Week 15–17 · 3 weeks · Needs: 4.2, 4.3

Do. Connect the microphone path to the engine. `host/demo_gui.py` shows the waveform, detected events, the predicted class, a confidence (softmax computed on the host from the logits) and the measured cycles.

Then finalise `docs/diagrams/system.drawio` so it matches what you actually built: every block, the clock-domain crossing marked, and both board wrappers. This is the replacement for synopsis Figure 5.1 and the figure you present at the defence.

Review gate.

- ☐ Recorded test gunshots, played through a speaker, are classified on screen.
- ☐ The demo runs for 30 minutes without a reset.
- ☐ The system diagram matches the built system, checked by a non-author, and is exported as SVG or PDF for the report.

Speaker playback changes the sound, so the live accuracy will be lower than the offline accuracy. That is expected. Report it separately as "live demo accuracy" and never mix it with the test-set number.

🏁 **Milestone B — "It works on hardware."**

STAGE 5 — HLS and Cadence (Weeks 3–19)

Lane. This stage runs alongside the others and is owned by whoever has capacity. **Core:** Steps 5.1 (decide the Cadence scope), 5.2 (HLS of layer 1) and 5.6 (RTL-vs-HLS comparison). **Stretch, only if the core is on track:** Steps 5.3 (Genus), 5.4 (formal proof) and 5.5 (HLS of the full model). The stretch steps are the first things to cut if the critical path slips.

What Cadence changes, and what it doesn't

Fill in Step 0.1 first. These are the possibilities, ranked by value to *your* project.

1. Genus + a PDK: an ASIC datapoint for the CIM tile. Highest value.

- Synthesizing `cim_tile` and the baseline datapath to a standard-cell library (for example GPDK045) gives real area and achievable clock at a process node.
- This directly answers the examiner's question "but it's just an FPGA": it shows the same RTL is retargetable to an ASIC flow, as the synopsis already promises.
- It is modest effort, *if* a PDK and a working setup exist.
- Synthesis only; place-and-route in Innovus is optional extra credit.

2. Xcelium: a better simulator. Faster runs on the full test set, and code-coverage reports. A coverage percentage is a concrete verification metric for the report.

3. JasperGold: a formal proof of the tile. Prove that `cim_tile` equals `W·a` for *every* possible input, not just 10,000 random ones. Even a proof on a reduced tile (4×4) is a strong result.

4. Stratus HLS: helps RTL-vs-HLS, with caveats.

- With Stratus and Genus you can synthesize the hand-written RTL and the Stratus-generated RTL to the *same* cell library. That is a cleaner comparison than on an FPGA, because it removes FPGA-specific mapping differences (LUTRAM vs BRAM, for example).
- But Stratus uses SystemC-style C++ and has a steeper learning curve than Vitis HLS. Learning two HLS tools is not worth it for a team of four.
- **Recommendation:** use Vitis HLS (free, accepts C or C++, targets your FPGA directly) for the RTL-vs-HLS comparison. Use Stratus only if Sir or a lab member already knows the flow and Genus + PDK are confirmed.

5. Things Cadence does not change.

- You will not design an actual SRAM CIM bitcell array in Virtuoso. That is custom analog/memory design, a separate research project in itself, and out of scope for this team and timeline.
- Joules can estimate power, but you dropped the energy claim. At most, report its estimate for context.

Step 5.1 — Decide the Cadence scope

Week 3 · meeting · Needs: 0.1

Using the confirmed tool and PDK list, pick which of points 1–4 above you will attempt. Record the decision.

Step 5.2 — HLS of layer 1

Week 14–16 · 3 weeks · Needs: 1.3, 1.5

Do.

1. Implement the real first layer (the Step 1.8 engine's job) in Vitis HLS in **plain C** (the team's choice). Use the standard types from `stdint.h`: `int8_t` for weights and activations, `int32_t` for accumulators, and `int64_t` for the requantization product (a 32-bit accumulator times a 16-bit multiplier needs more than 32 bits).
   - **Why not `ap_int`.** The current Vitis HLS guide documents its arbitrary-precision types (`ap_int`, `ap_fixed`) for C++ only, and the old C versions (`ap_cint.h`) can't be used with standard C compilers. So plain C means standard widths. Hand-written RTL can use exact widths (a 12-bit partial sum, a 16-bit multiplier); C will use 16 or 32 bits wherever the tool can't prove a narrower one is safe, so the HLS version may use more LUTs for that reason alone. Report that as a fact about the method ("plain C, standard integer types"), not as a flaw in the tool. If you want to separate the effect, add one extra C++ row using `ap_int` with exact widths.
   - **Confirm that your installed Vitis HLS version accepts a `.c` top-level file**, and keep to what C allows: no templates, no references (pass pointers or arrays), no `hls::stream` (C++ only; use arrays).
   - **Two C traps.** Right-shifting a negative signed integer is implementation-defined in C. The spec requires an arithmetic shift, so test negative values explicitly in the C simulation. And signed overflow is undefined behaviour, which is why the accumulator bound from `arith_spec.md` must hold.
2. Reuse the layer-1 test vectors from Step 1.5.
3. Compare against the RTL layer-1 engine on: LUT, FF, DSP, f_max, cycles, and **engineering hours** (log your hours from day one).

Review gate.

☐ The HLS layer 1 is bit-exact on every layer-1 test input, including inputs that make the accumulator and the requantization product negative.
☐ The language and integer types used are written in the comparison table (plain C, `stdint.h` types).
☐ A first RTL-vs-HLS comparison table exists for layer 1.
☐ The DSP count is reported for the whole design, and compared with the RTL's whole-design number like for like. If multiplies were forced into logic, the directive used is recorded.

LLM help. Sonnet writes the HLS code. **Verify every `#pragma HLS` against the Vitis HLS user guide (UG1399).** Models invent pragmas, and a wrong pragma is silently ignored.

Step 5.3 — Genus synthesis of the tile (if confirmed)

Week 13–14 · stretch · Needs: 1.6, 5.1

Do. Synthesize `cim_tile` (and later the baseline datapath) in Genus with the PDK. Record area, the timing at the target clock, and the critical path.

An ASIC flow has no LUTRAM. Without an SRAM memory compiler, Genus builds the weight storage from flip-flops, so this is a *register-file* CIM tile, not an SRAM one. Report storage area and compute area separately, and name it that way in the report.

Review gate.

☐ A gate-level simulation of the Genus netlist passes the tile testbench.
☐ Conformal LEC passes between RTL and netlist, if available.

Step 5.4 — Formal proof of the tile (if JasperGold is confirmed)

Week 18–19 · stretch goal (only if everything else is on track; nothing depends on it)

Do. Write assertions stating that the output equals the reference dot product. Prove them on a 4×4 tile first, then scale up as far as the tool allows.

Step 5.5 — HLS of the real model

Week 17–19 · stretch · Needs: 2.7, 5.2

Do. Implement the real model in HLS, in plain C with the same conventions as Step 5.2, and run a reuse-factor sweep. Reuse the C testbench and the test-vector reader from Step 5.2.

Review gate.

☐ Bit-exact on the 200 clips.
☐ The comparison table against the RTL is filled in, with the language and types stated.

Step 5.6 — RTL vs HLS comparison write-up

Week 17–18 · 2 weeks · Needs: 5.2, 4.1

Do. Write it up from `runs.csv` and the hours log.

STAGE 6 — Measurement, then claims (Weeks 13–17)

Step 6.1 — Write the measurement protocol before measuring

Week 13 · 3 days · Needs: 3.2, 1.11

Do. Write `docs/measurement_protocol.md`, defining:

- **L_inf (inference latency):** cycles from the last sample of the window being present in the input buffer to the class being valid at the output, × the clock period.
- **II (initiation interval):** cycles between successive inference starts at steady state.
- **L_e2e:** acquisition + preprocessing + L_inf. This is reported for realism; the claim is not built on it.
- How each quantity is measured: the on-chip counter, cross-checked against post-route simulation and an oscilloscope on a toggle pin.
- How many runs, and what is reported (median, plus min/max).
- The configurations and clock frequencies.
- **The board.** Headline measurements are taken on the platform chosen in Decision D14. The other board is reported as a second platform where the design fits. Numbers from different boards are never compared with each other.

Review gate.

☐ Sir approves the protocol **before** any Stage 6 measurement is taken.

Step 6.2 — Run the measurements

Week 15–16 · 7 days · Needs: 6.1, 4.2

Do. Run every configuration on the board per the protocol. Everything goes into `runs.csv`.

Step 6.3 — Optional microcontroller baseline

Week 18–19 · stretch · Needs: 2.6

Do. Run the same quantized model on a Cortex-M4 board using a **plain-C implementation of the integer forward pass**, ported from `ref_int.py` and `arith_spec.md`. The C code from Step 5.5 is a good starting point because both are plain C. This is a better baseline than TFLite-Micro (which is C++ and uses its own quantization scheme), because it computes exactly your arithmetic. Clearly label it in the report as a *different class of device*.

Step 6.4 — Compare predictions with measurements, then write claims

Week 16–17

Do.

1. Plot predicted (3.3) vs simulated (3.7) vs measured (6.2) cycles.
2. Plot measured cycles against `n_macs` for both engines. This is the plot that tests the hypothesis.
3. **Only now**, write the claims. Each claim sentence states what was measured, under which configuration, on which board, and cites the `runs.csv` rows it comes from.
4. If the hypothesis was not supported, or only partly, report exactly that. "The baseline saturated at 32 MACs with a 64-bit weight port; CIM continued scaling to 128" is a claim. "CIM is 5× faster" without the configuration is not.

Review gate.

☐ Every number in the results chapter traces to a `runs.csv` row.
☐ GPT, prompted as a hostile examiner, is given the claims and the protocol. Every objection is answered.

STAGE 7 — Demo polish, report, defence (Weeks 6–20)

Write the report from Week 2, not Week 16. Each step's spec document (`arith_spec`, `mapping_spec`, `baseline_spec`, `measurement_protocol`) is most of a methodology chapter already.

Week	Step
6–15	Methodology chapters drafted from each spec document as it passes its gate (about half a day per spec).
16–17	Results chapter generated from `runs.csv`. Figures produced by scripts.
17–18	Full draft. Each member reviews a chapter they did not write.
18–19	Structural review (Opus) and language polish (GLM or Sonnet).
19	Mock defence. Each member presents a part they did *not* build, and someone outside the team attacks it.
19–20	Fixes, slides, demo rehearsal × 3.
20	Submit and defend.
21–24	**Buffer.** Nothing is planned here. This is what absorbs a slip; the end of March is the hard limit.

The critical path, at a glance

```
0.1 → 0.3 → 1.1 → 1.2 → 1.3 → 1.6 → 1.8 → 1.9 → 1.10 ─┐
            1.7 board link, alongside, joins at 1.10  │
0.4 → 2.1 → 2.3 → 2.4 → 2.5 → 2.6 → 2.7 → 2.8 ────────┤
                                                      ▼
                    3.1 → 3.4 → 3.5 → 4.1 → 4.2 → 6.2 → 6.4 → 7.1 → 7.2 → 7.3 → 7.4 → 7.5 → 7.6
                    3.2 → 3.3, 6.1 (protocol, before 6.2);   3.6 ┘  3.7 runs alongside 4.1;  4.3 → 4.4 joins at 7.5
Off critical path (cut first if late): Steps 5.3, 5.4, 5.5 and 6.3
```

Effort check: the plan totals about 308 person-days (one day = 6 focused hours). About 263 are core; about 45 are stretch (Steps 5.3, 5.4, 5.5 and 6.3, marked "cut first"). Four people at 25 hours a week give about 333 person-days in 20 weeks, so the core uses about 79% of capacity and the stretch only fits if everything else is on track. Weeks 5, 11 and 16–17 run at about 110–120% of capacity, so protect those weeks from coursework. Weeks 21–24 are held as buffer up to the end of March. **If the real hours turn out nearer 20 per week, the core still just fits in 20 weeks with no slack, so drop the stretch items at once and lean on the buffer. If you are more than 2 weeks behind at Milestone A (Week 8), drop all stretch items and cut Stage 5 to "HLS of layer 1 only" immediately.**

Which LLM for what

Job	Model	Why
Writing code from a clear spec (RTL, testbenches, Python, TCL, plotting)	**Sonnet**	Most of your volume. Mechanical work with a clear specification.
Spec documents, arithmetic and quantization math, architecture and baseline fairness, bugs that survive an hour	**Opus**	Where a wrong answer costs a rework cycle.
A bug Opus has failed on twice, or a costly one-way decision	**Fable**	Rarely; probably 3–4 times all year.
Huge inputs: full Vivado/Genus logs, timing reports, datasheets, several papers at once	**Gemini**	Long context. Use it to extract and summarize, not to design.
Attacking your claims, specs and baseline as a hostile examiner	**GPT**	A model outside the conversation finds what you've stopped seeing.
Boilerplate, docstrings, LaTeX formatting, language polish	**GLM**	Cheap and adequate for non-critical text.

Rules:

1. The integer reference and the RTL testbench are written **in separate conversations, from the spec document only**. Never let one model see the other's code while writing.
2. **No LLM writes a number into results.** Every number comes from `runs.csv`.
3. **Verify every vendor-specific detail against the official documentation:** Vivado TCL options, XDC constraints, `#pragma HLS`, pin assignments. Models invent these, and they fail silently.
4. **Clock-domain-crossing logic comes from a proven template**, never written fresh by a model.
5. **Start a fresh conversation for each step**, and paste in the relevant spec. Long threads get slower, costlier and less accurate.

Data sources (verify before citing)

Primary dataset: provided by Sir. Cite it as he directs. The public datasets below are for pipeline development before it arrives, background-class hard negatives, the fallback workload and a cross-dataset test.

- R. Kabealo et al., "A multi-firearm, multi-orientation audio dataset of gunshots," *Data in Brief*, vol. 48, 109091, 2023. Dataset: Zenodo DOI 10.5281/zenodo.7004819 (CC-BY-4.0).
- S. Gurny, R. Quinn, "Descriptor: Certus Caliber Classification Gunshot Dataset (C3GD)," arXiv:2606.18135, 2026. Field-collected, 28 firearms, 16 calibers; check download availability.
- "Deciphering GunType Hierarchy through Acoustic Analysis of Gunshot Recordings," arXiv:2506.20609 (CMU). Five broad gun-type categories, joint detection and classification.
- UrbanSound8K, ESC-50 (background class); Google Speech Commands (fallback workload).
- BGG in-game gunshot dataset, arXiv:2210.05917. Synthetic; only ever an optional pretraining aid, never test data.

Monday morning

1. Ask Sir the eight questions in **Step 0.1**.
2. Start **Step 0.2** (objectives) and **Step 0.4** (dataset download) the same day.
3. Whoever sets up the repo (**Step 0.3**) makes the blinking LED build from the command line by Friday.
4. Week 2 begins **Step 1.2**, the arithmetic spec. That document is the real start of the engineering.

For whoever runs individual steps from this plan: Sonnet for most steps, with the relevant spec pasted into a fresh conversation. Opus for Steps 0.2, 1.2, 1.3, 2.6, 3.1, 3.2 and any bug older than an hour. Default thinking throughout. Extended thinking only for Steps 1.2, 2.6 and 3.2 (arithmetic, quantization scaling, baseline fairness).